

Is Bash All You Need?

An Empirical Study of Tool Interfaces for Enterprise Digital Worker Agents

Hazel Mak^{1*}, Susheel Suresh¹, Sahil Bhatnagar¹,
Barry Wang², Chhaya Methani¹, Alejandro Gutierrez Munoz¹

¹Microsoft Corporation
One Microsoft Way, Redmond, WA 98052, USA

²Carnegie Mellon University
5000 Forbes Avenue, Pittsburgh, PA 15213, USA

Abstract

In this study, we examine whether a general shell can outperform specialized tools on enterprise tasks. Shell-based agents have shown strong results in coding, but enterprise work also involves moving between applications and services, coordinating with coworkers, and performing professional analysis. We compare five tool interfaces on TheAgentCompany and APEX-Agents using Opus-4.8 and GPT-5.5: typed tools, typed tools plus bash, bash alone, bash with persistent agent-synthesized tools, and programmatic tool calling (PTC), which runs programs whose actions are restricted to a typed tool catalog. Bash alone outperforms typed tools on both benchmarks, improving score by 21.8–24.5 pp on TheAgentCompany and 4.8–7.4 pp on APEX-Agents while using 19–72% fewer total tokens. Adding typed tools or persistent tool synthesis to bash produces no detectable pooled score gain. PTC uses fewer tokens than direct typed calls with broadly similar task performance, but generally underperforms bash alone in both quality and cost efficiency. For enterprise practitioners, these results favor bash alone when arbitrary execution can be isolated and PTC when security or compliance policies require a fixed tool catalog.

1 Introduction

Can a shell outperform specialized tools on enterprise agent tasks? Frontier models are trained to use bash as a native action language. In coding, mini-SWE-agent exposes only bash (Lieret, Yang, and agent Team 2025), while Live-SWE-agent topped SWE-bench Verified by adding agent-created tools (Xia et al. 2025).

Enterprise products are adopting both catalog-based and shell-based orchestration. Anthropic and OpenAI offer programmatic tool calling over typed catalogs through their APIs (Anthropic 2025b; OpenAI 2026b). Shell-based offerings include Anthropic’s Claude Code (Anthropic 2025a), OpenAI’s Codex CLI and SDK (OpenAI 2026a), and Microsoft’s Copilot Studio, which uses the GitHub Copilot CLI and SDK (Microsoft 2026).

However, controlled comparisons of shell execution and typed tool interfaces on enterprise workflows remain limited, leaving practitioners with little evidence to guide interface selection.

We conduct a controlled tool-interface ablation on two enterprise workplace benchmarks: **TheAgentCompany** (Xu et al. 2024), spanning software company roles, and **APEX-Agents** (Vidgen et al. 2026), covering investment banking, management consulting, and corporate law. We compare five interfaces: **TOOL-ONLY**, **BASH+TOOL**, **BASH**, **BASH+SYNTHESIS**, and programmatic tool calling (PTC) (Anthropic 2025b; OpenAI 2026b).

Across both benchmarks and models, **BASH** outscores both shell-free interfaces while remaining among the cheapest options. Adding typed or synthesized tools to **BASH** yields no detectable pooled score gain. PTC uses fewer tokens than **TOOL-ONLY** but underperforms **BASH** in task score and token efficiency.

Our contributions are twofold: (i) a controlled comparison of tool interfaces spanning direct tool calling and code-based orchestration; and (ii) an evaluation of their quality and cost on realistic enterprise workflows across diverse professional roles.

2 Related Work

From tool calls to executable actions. Typed tool calling gives agents access to external functions (Yao et al. 2022; Schick, Dwivedi-Yu et al. 2023; Patil et al. 2023), with function-call capability evaluated by BFCL (Patil et al. 2025). Code-based approaches extend this interaction by expressing reasoning (Gao et al. 2023; Chen et al. 2022), actions (Wang et al. 2024, 2025), and control flow (Kim et al. 2024; Lumer et al. 2026; Qi et al. 2026) in executable programs. Programmatic tool calling applies this idea to a fixed catalog of tools (Anthropic 2025b; OpenAI 2026b). Together, these approaches motivate our comparison of direct calls, catalog-constrained programs, and shell execution.

Shell-based agents and tool synthesis. Coding agents illustrate the range of interface choices: specialized tools in SWE-agent (Yang et al. 2024) on SWE-bench (Jimenez, Yang et al. 2023; OpenAI 2024), shell-only execution in mini-SWE-agent (Lieret, Yang, and agent Team 2025), self-authored tools in Live-SWE-agent (Xia et al. 2025), and a fixed workflow in Agentless (Xia et al. 2024). Tool synthesis and persistent skill libraries further allow agents to package capabilities for reuse (Cai et al. 2023; Qian et al. 2023; Wang et al. 2023; Yuan et al. 2023; Wang, Fried, and

*Corresponding author: hazelmak@microsoft.com

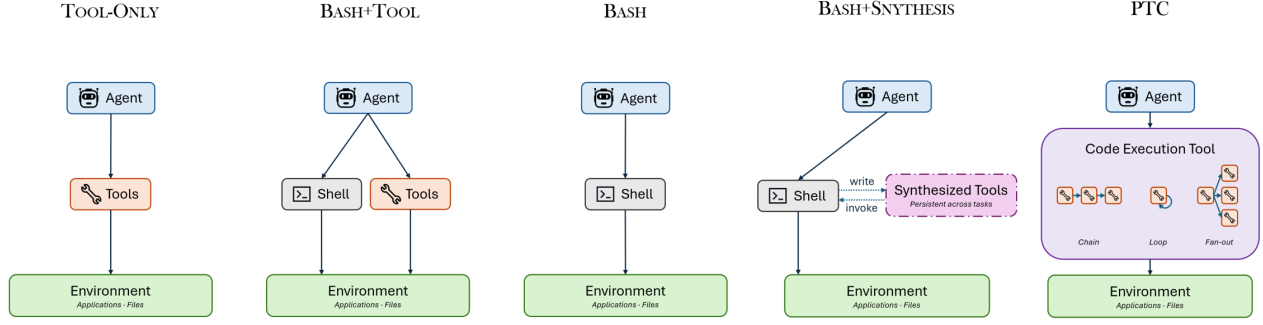
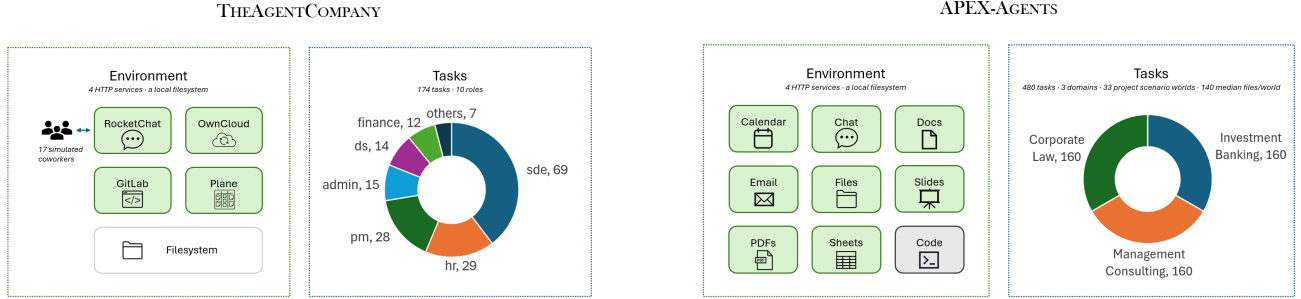


Figure 1: Five interfaces formed from shell execution, typed tools, persistent synthesized tools, and restricted programs over typed tools.



(a) TheAgentCompany: 4 self-hosted services, a local workspace, and 17 simulated coworkers reachable through RocketChat.

(b) APEX-Agents: 9 MCP servers over a shared workspace, with tasks organized into 33 scenario worlds.

Figure 2: Benchmark environments and task types.

Neubig 2024; Liu et al. 2026), including extending available catalogs (Haque et al. 2025) and supporting computer control (Wu et al. 2024). These designs motivate testing whether typed tools or persistent synthesized tools add value to a shell-based enterprise agent.

Evaluation beyond function calling. Enterprise benchmarks evaluate agents on customer service (Yao et al. 2025), web applications (Drouin et al. 2024), desktop tasks (Xie et al. 2024), and workplace workflows (Xu et al. 2024; Vidgen et al. 2026). Concurrently, Patel et al. (2026) compares programmatic and native JSON tool calling on a BFCL v4 subset. Our independent study instead compares shell execution, direct tool calling, and programmatic tool calling on realistic enterprise workflows.

3 Method

An agent’s *tool interface* defines how it can act on its environment, shaping its capabilities, deployment safety, and operating cost. A shell permits arbitrary code execution, allowing many actions in one step but requiring controls on execution and access. A curated catalog instead exposes only deployer-authorized functions, with overhead when called individually. Programmatic tool calling (Anthropic 2025b; OpenAI 2026b) combines these calls in a program that can

chain, loop over, and parallelize them while restricting environment actions to the catalog.

We compare five tool interfaces (Figure 1) with the same model, task text, base prompt, and stopping criteria, adding interface-specific prompts for synthesis and PTC. **BASH** exposes only the shell, **TOOL-ONLY** exposes only the typed catalog, and **BASH+TOOL** gives the agent both the shell and the catalog. **BASH+SYNTHESIS** starts with shell access and a persistent directory. Tasks run sequentially, with prompts encouraging the agent to create reusable tools and invoke them on later tasks to reduce discovery costs. **PTC** lets the model write a restricted program over the typed catalog. Our implementation uses Python as the program language and exposes the catalog as ordinary functions within a restricted runtime that provides no shell or other action channel.

4 Experiment Setup

We run evaluations on TheAgentCompany and APEX-Agents under all five interfaces with **Opus-4.8** and **GPT-5.5** at default reasoning levels.

4.1 TheAgentCompany

TheAgentCompany (Xu et al. 2024) simulates a software company where agents use shared services, work with local

Interface	Model	Score (95% CI)	Pass rate (95% CI)	Input	Output	Cache-read	Total	\$/task	Time/task
<i>TheAgentCompany</i>									
TOOL-ONLY	Opus-4.8	44.9% [38.7%, 51.0%]	24.1% [17.8%, 30.5%]	929k	16k	855k	945k	\$1.20	5.1m
	GPT-5.5	45.3% [39.3%, 51.3%]	25.3% [19.0%, 31.6%]	458k	7k	408k	465k	\$0.66	1.8m
BASH+TOOL	Opus-4.8	67.2% [61.3%, 73.0%]	50.0% [42.5%, 57.5%]	563k	6k	537k	569k	\$0.55	3.1m
	GPT-5.5	66.3% [60.3%, 72.2%]	47.7% [40.2%, 55.2%]	503k	6k	462k	508k	\$0.60	2.5m
BASH	Opus-4.8	69.4% [63.7%, 74.9%]	50.6% [43.1%, 58.0%]	258k	6k	239k	264k	\$0.37	3.0m
	GPT-5.5	67.1% [61.4%, 72.7%]	47.7% [40.2%, 55.2%]	370k	7k	331k	377k	\$0.56	2.6m
BASH+SYNTHESIS	Opus-4.8	67.1% [61.2%, 72.7%]	47.7% [40.2%, 55.2%]	244k	5k	224k	249k	\$0.35	3.1m
	GPT-5.5	66.4% [61.0%, 71.8%]	45.4% [37.9%, 52.9%]	429k	7k	388k	436k	\$0.61	3.0m
PTC	Opus-4.8	52.9% [46.8%, 59.1%]	29.9% [23.0%, 36.8%]	395k	12k	362k	406k	\$0.64	3.2m
	GPT-5.5	45.1% [39.2%, 51.2%]	24.1% [17.8%, 30.5%]	232k	10k	207k	242k	\$0.54	1.9m
<i>APEX-Agents</i>									
TOOL-ONLY	Opus-4.8	41.1% [37.4%, 44.8%]	24.2% [20.4%, 28.1%]	384k	13k	338k	397k	\$0.73	3.4m
	GPT-5.5	39.8% [36.1%, 43.4%]	23.3% [19.6%, 27.1%]	322k	6k	282k	328k	\$0.52	2.1m
BASH+TOOL	Opus-4.8	47.2% [43.5%, 50.8%]	29.8% [25.6%, 34.0%]	386k	10k	350k	396k	\$0.60	3.0m
	GPT-5.5	45.5% [41.8%, 49.3%]	29.6% [25.6%, 33.5%]	253k	5k	218k	257k	\$0.42	2.0m
BASH	Opus-4.8	48.5% [44.9%, 52.2%]	30.0% [26.0%, 34.2%]	205k	10k	177k	215k	\$0.48	3.5m
	GPT-5.5	44.6% [40.8%, 48.3%]	28.3% [24.2%, 32.5%]	171k	5k	137k	176k	\$0.40	2.4m
BASH+SYNTHESIS	Opus-4.8	46.9% [43.3%, 50.7%]	28.3% [24.4%, 32.5%]	226k	11k	201k	237k	\$0.50	3.7m
	GPT-5.5	45.9% [42.2%, 49.7%]	29.2% [25.2%, 33.1%]	222k	5k	186k	228k	\$0.44	6.6m
PTC	Opus-4.8	40.1% [36.5%, 43.7%]	22.3% [18.5%, 26.2%]	324k	12k	294k	337k	\$0.61	3.4m
	GPT-5.5	37.8% [34.2%, 41.5%]	20.6% [17.1%, 24.4%]	251k	5k	207k	256k	\$0.48	2.1m

Table 1: *Score*: effort-weighted checkpoint rate (TheAgentCompany) or mean rubric score (APEX-Agents); *pass rate*: fully solved fraction; both with 95% bootstrap CIs. Token columns: means/task in thousands; input includes cache-read; total = input + output. *\$/task*: mean estimated inference cost at GitHub Copilot list rates; *time/task*: mean wall-clock minutes.

files, and collaborate with simulated coworkers to complete coding and office workflows (Figure 2a).

The benchmark provides no LLM-facing typed tool catalog. We construct a task-oriented catalog of 60 typed tools covering service APIs and workspace operations across the 174 evaluated tasks. The catalog includes 54 service tools and six workspace file tools (Appendix B.1).

TheAgentCompany coworkers are simulated by GPT-5. We patch coworker reliability defects in the benchmark release: missed messages, premature termination, and stale state. The fixes apply uniformly across interfaces. Grading uses the benchmark’s Python checkpoint evaluators.

4.2 APEX-Agents

APEX-Agents (Vidgen et al. 2026) evaluates professional analysis in investment banking, management consulting, and corporate law. Agents use shared-workspace files to answer questions or create and edit documents, spreadsheets, and presentations (Figure 2b).

We evaluate all 480 tasks using a fixed June 2026 Archipelago snapshot with nine MCP servers, rather than the later eleven-server version, to avoid reliance on paid financial-data APIs. We retain its 20 typed application tools and use its code-execution server as BASH: a guardrailed Unix shell over the workspace (Appendix B.2).

An Archipelago-derived LLM judge scores rubric criteria using the final response and workspace-file changes.

5 Results

5.1 Core Outcomes

We compare task quality, token use, and inference cost across interfaces (Table 1; Figure 3). BASH outscores both shell-free interfaces in every benchmark-model arm while remaining among the cheapest options. Compared with TOOL-ONLY, it uses fewer tokens and lowers estimated inference cost throughout.

PTC consistently saves tokens over TOOL-ONLY, but improves score only on TheAgentCompany with Opus-4.8; elsewhere scores are similar or slightly lower. It scores below BASH throughout, with a token advantage only on TheAgentCompany with GPT-5.5.

Adding typed or synthesized tools to BASH generally increases token use without a detectable pooled score improvement in paired comparisons (Appendix C.2). The sole token-use exception is BASH+SYNTHESIS on TheAgentCompany with Opus-4.8.

Benchmark differences are clearest in BASH’s larger pass-rate gains over TOOL-ONLY on TheAgentCompany, despite similar baselines on both benchmarks. Flexible execution may help more with coding and cross-application workflows than with domain reasoning, consistent with the domain breakdown and failure analysis (Appendices C.1 and C.5).

Model differences suggest workload-specific choices. On TheAgentCompany, Opus-4.8 scores higher with fewer tokens when using the shell without a typed catalog, favoring it for similar coding and workflow tasks; its PTC score ad-

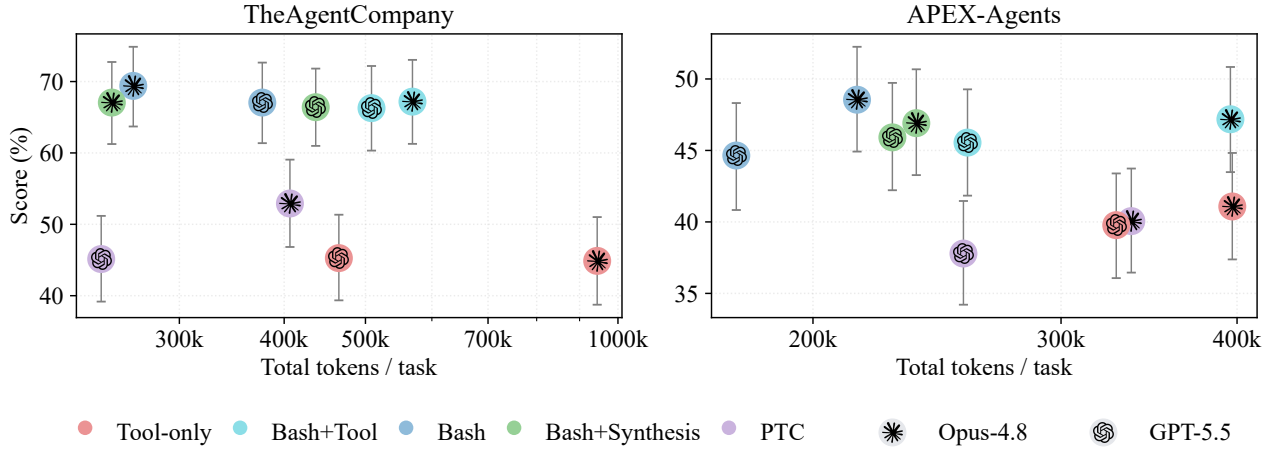


Figure 3: Score (%) versus total tokens/task (log scale). Logos: models; fills: interfaces; vertical bars: 95% bootstrap CIs. Higher-left is better.

vantage requires more tokens. GPT-5.5 is a token-efficient choice for direct typed-tool access, achieving similar scores on both benchmarks. Document-heavy professional analysis presents a tradeoff: across all APEX-Agents interfaces, Opus-4.8 scores higher while GPT-5.5 uses fewer tokens.

5.2 Tool Use

We compare number of tool calls, total tool-call tokens, tool call success rate, and exact repetition across interfaces (Table 2). BASH reduces number of tool calls and tool-call tokens (more than halved on TheAgentCompany) relative to TOOL-ONLY, with similar call-success rates.

Interface	# Tool calls /task	Tool tokens /task	Success	Repeated
<i>TheAgentCompany</i>				
TOOL-ONLY	14.0	19.6k	93.3%	16.2%
BASH+TOOL	13.5	11.3k	91.5%	4.0%
BASH	12.8	9.0k	92.1%	0.3%
BASH+SYNTHESIS	14.2	9.7k	92.1%	0.3%
PTC	8.8	8.4k	65.2%	3.8%
<i>APEX-Agents</i>				
TOOL-ONLY	13.0	15.6k	90.9%	1.7%
BASH+TOOL	11.5	13.8k	90.7%	0.4%
BASH	7.5	10.4k	91.1%	0.0%
BASH+SYNTHESIS	9.5	12.1k	87.7%	0.2%
PTC	9.5	12.8k	86.2%	0.2%

Table 2: Tool use statistics. # *Tool calls/task*: median tool calls/task. *Tool tokens/task*: median tool-name, argument, and result tokens/task; excludes context/cache effects and is not billed usage. Internal Bash/PTC operations excluded; only returned results count. *Success*: calls reporting success; *repeated*: exact tool-and-argument repeats within a task.

Adding typed tools or synthesis to BASH increases calls and tool-call tokens without consistently improving call re-

liability.

PTC has the lowest tool-call success rate on both benchmarks. This may reflect the added difficulty of coordinating tool calls and handling intermediate failures within generated programs.

TOOL-ONLY has the highest exact-repeat rate on both benchmarks. This may be because shell-based and programmatic interfaces can bundle repeated operations into one invocation, whereas TOOL-ONLY exposes each repetition as a separate tool call.

5.3 Paired Performance by Task Complexity

We examine how BASH and PTC score gains over TOOL-ONLY vary with task complexity (Figure 4). We use task-model paired score differences, binning tasks by their two-model average TOOL-ONLY call count as a complexity proxy.

BASH’s score gains are larger on TheAgentCompany than on APEX-Agents. On TheAgentCompany, its advantage peaks at intermediate counts, while PTC shows smaller, uneven gains. On APEX-Agents, both interfaces gain more at high counts, where PTC’s mean score difference becomes positive and BASH retains the larger advantage.

Two limitations qualify this analysis. First, PTC’s confidence intervals include zero in most bins, limiting evidence for a consistent score gain. Second, TOOL-ONLY call count is an imperfect complexity proxy: high counts can reflect interface inefficiency rather than intrinsic task difficulty.

5.4 Bash Efficiency

We examine bash efficiency via composition, code length, failures, and immediate same-command retries (Table 3). Composition remains the norm across all three shell interfaces, even when typed or synthesized tools are available.

BASH+TOOL reduces shell use without improving the reliability of remaining shell calls. It combines the lowest median shell calls per shell-using task with the highest failure rate and lowest retry rate in both benchmarks (Appendix C.3).

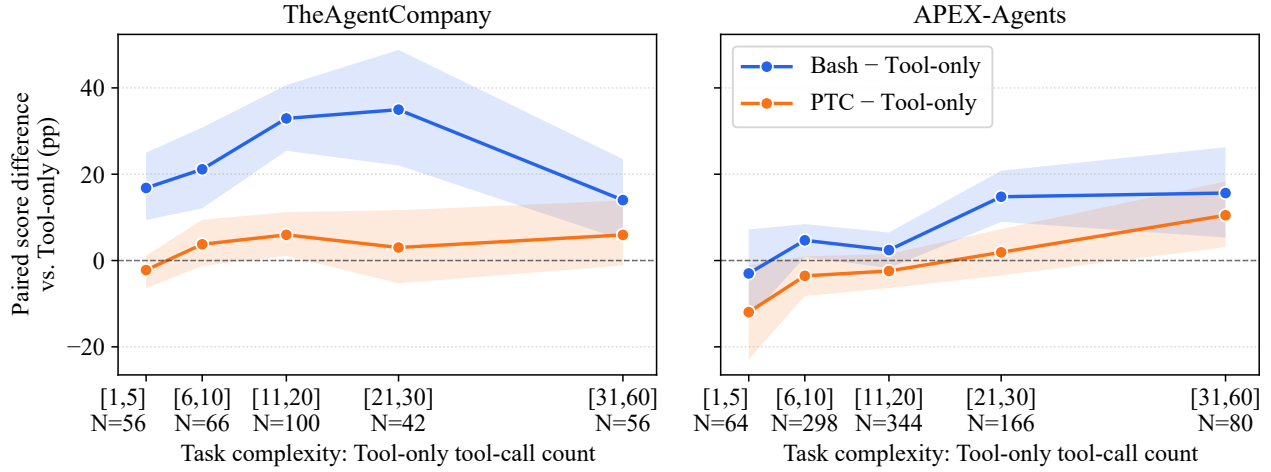


Figure 4: Mean paired score differences (pp) from TOOL-ONLY: BASH (blue), PTC (orange). Bins: mean TOOL-ONLY calls across models; bands: 95% bootstrap CIs; N : task-model pairs/bin. Budget-clipped counts are lower bounds.

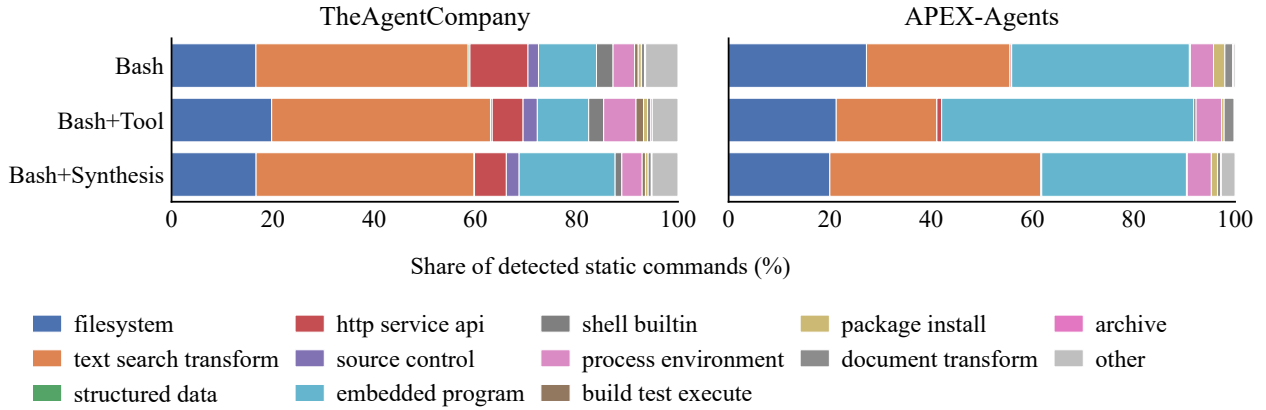


Figure 5: Static shell-command shares by executable family, pooled across models (Table 11). Bars: 12 families plus *other*, summing to 100% per benchmark/interface.

Interface	Cmds /call	LOC	Composed	Loop	Failed	Repair
<i>TheAgentCompany</i>						
BASH	4.0	2	94.3%	27.5%	7.9%	76.3%
BASH+TOOL	3.0	2	93.9%	25.7%	10.8%	58.7%
BASH+SYNTHESIS	3.0	1	88.0%	24.0%	7.9%	70.0%
<i>APEX-Agents</i>						
BASH	1.5	12	96.5%	67.8%	4.4%	55.8%
BASH+TOOL	1.5	16	97.2%	74.0%	9.3%	35.8%
BASH+SYNTHESIS	2.0	5	93.3%	48.2%	5.4%	51.1%

Table 3: Shell statistics. *Cmds/call*: median commands. *LOC*: lines of code. *Composed*: calls with pipe, logical-operator, substitution, control-flow, heredoc, multiline, or nontrivial-redirect markers; *loop*: control-flow markers. *Failed*: failures. *Repair*: failures whose next call shares the first command.

BASH+SYNTHESIS packages code behind shorter invocations, reflected in its lowest median lines of code (LOC) and composition fraction on both benchmarks. Calls invoking synthesized tools are also shorter on average than other synthesis calls, supporting code packaging as a practical benefit (Appendix C.3).

Benchmark differences reveal two uses of the shell. TheAgentCompany combines more static commands into short calls, with text utilities and HTTP clients more prominent (Figure 5). APEX-Agents uses longer embedded programs; most control-flow flags disappear when embedded and quoted bodies are stripped. Shell access can therefore support both utility-based workflows and file analysis.

5.5 Tool Synthesis

We examine whether synthesized tools are reused reliably and whether their use is associated with better quality or lower token use. Agents invoke most tools they create, but

fewer reach later tasks (Table 4). On APEX-Agents, GPT-5.5 creates fewer tools than Opus-4.8 yet uses all in later tasks. Appendix C.4 details the tools and measurements.

Reuse does not consistently improve call success. Reused tools and fresh inline code have similar success rates with Opus-4.8, but reused tools fare worse with GPT-5.5.

Tasks that invoke synthesized tools show no detectable mean score gain over BASH on either benchmark (Table 5). On APEX-Agents, token differences relative to BASH are larger in the invoked subset than in the non-invoked subset.

Model	Created	Invoked	Cross-task reuse	Reuse success	Fresh success
<i>TheAgentCompany</i>					
Opus-4.8	21	19 (90.5%)	12 (57.1%)	94.0%	95.3%
GPT-5.5	12	10 (83.3%)	6 (50.0%)	83.7%	88.3%
<i>APEX-Agents</i>					
Opus-4.8	32	29 (90.6%)	14 (43.8%)	96.4%	96.1%
GPT-5.5	10	10 (100.0%)	10 (100.0%)	80.1%	86.2%

Table 4: Tool synthesis reuse statistics and success rates. *Created*: distinct agent-authored root Python paths, excluding seeds, initializers, and dependencies. *Invoked*: tools used from creation onward; *cross-task reuse*: used in later tasks. Percentages divide by created tools. *Reuse success*: successful calls of synthesized tools; *fresh success*: successful calls of fresh shell commands.

Subset	<i>N</i>	Median Δ score	Mean Δ score	Median Δ tokens	Mean Δ tokens
<i>TheAgentCompany</i>					
Invoked	228	+0.0 pp	-0.6 pp	+0.6k	+1.1k
Not invoked	120	+0.0 pp	-3.7 pp	+0.8k	+3.3k
<i>APEX-Agents</i>					
Invoked	559	+0.0 pp	+0.1 pp	+31.4k	+63.0k
Not invoked	401	+0.0 pp	-0.6 pp	+8.5k	-0.3k

Table 5: Paired differences of BASH+SYNTHESIS minus BASH, pooled across models. *Subset*: whether synthesized tools are invoked; *N*: number of tasks in the corresponding subset. Δ score: task-score difference; Δ tokens: total-token difference.

5.6 Programmatic Tool Calling

We compare PTC with TOOL-ONLY to assess call batching and changes in agent-visible invocations and token use (Table 6). Despite modest batching and invocation reductions, PTC substantially reduces uncached input and cache-read tokens on both benchmarks, with broadly similar output-token use. These savings may reflect processing intermediate results within programs and returning only selected outputs, reducing the context carried into subsequent model requests.

6 Discussion

Shell-only agents offer the strongest overall task performance and cost efficiency with the simplest tool scaffold in this

Benchmark	Comp. ratio	Δ call	Δ uncached input	Δ cache-read	Δ output
TheAgentCompany	2.0	-4.0	-97.6k	-86.8k	+0.2k
APEX-Agents	2.0	-3.0	-45.4k	-44.0k	-0.4k

Table 6: Programmatic tool calling composition and token efficiency. *Comp. ratio*: per-task typed calls per PTC execution. Deltas: PTC minus TOOL-ONLY, paired by task and model; tool calls count; uncached input tokens; cache-read tokens; output tokens.

study. Adding a typed catalog or persistent synthesized-tool library to the shell provides no consistent quality gain. Programmatic tool calling reduces token use over direct typed-tool calling with broadly similar or slightly better performance depending on workplace scenarios, while underperforming shell-only on both quality and cost. Practitioners could use shell-only agents where arbitrary execution can be isolated, and programmatic tool calling where security or compliance requirements restrict actions to an approved catalog.

Our results suggest starting with Opus-4.8 for shell-only coding and cross-application work, or GPT-5.5 for token-efficient direct tool calls. For document-heavy analysis and programmatic tool calling, practitioners could weigh Opus-4.8’s higher quality against GPT-5.5’s lower token use. As frontier models evolve, these recommendations may not carry over to later releases in either family.

Enterprise platforms could expose direct typed-tool calling, shell-only execution, and programmatic orchestration restricted to an approved catalog as configurable options. Practitioners could select among them according to task needs and whether general code execution is permitted.

Platforms could provide easy-to-navigate evaluation infrastructure that lets practitioners A/B-test interfaces and models on their own workplace scenarios. With permissions and human review held constant, teams could compare how often outputs meet users’ needs, how much manual revision they require, operating cost, and operational failures. These evaluations could help practitioners choose the simplest tool interface that meets their workplace’s quality, cost, and permission requirements.

Acknowledgments

We thank the Microsoft Copilot Studio team for their thoughtful feedback and constructive discussions throughout this work. We are especially grateful to Soufiane Loukili for his sponsorship and continued support of this research.

References

- Anthropic. 2025a. Claude Code. <https://code.claude.com/docs/en/overview>. Accessed: 2026-08-06.
- Anthropic. 2025b. Introducing advanced tool use on the Claude Developer Platform. Anthropic Engineering blog.
- Cai, T.; Wang, X.; Ma, T.; Chen, X.; and Zhou, D. 2023. Large Language Models as Tool Makers. *arXiv preprint arXiv:2305.17126*.
- Chen, W.; Ma, X.; Wang, X.; and Cohen, W. W. 2022. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *arXiv preprint arXiv:2211.12588*.
- Drouin, A.; Gasse, M.; Caccia, M.; Laradji, I. H.; Del Verme, M.; Marty, T.; Vazquez, D.; Chapados, N.; and Lacoste, A. 2024. WorkArena: How Capable Are Web Agents at Solving Common Knowledge Work Tasks? In *International Conference on Machine Learning (ICML)*.
- Gao, L.; Madaan, A.; Zhou, S.; Alon, U.; Liu, P.; Yang, Y.; Callan, J.; and Neubig, G. 2023. PAL: Program-aided Language Models. In *International Conference on Machine Learning (ICML)*, 10764–10799.
- Haque, M. A.; Williams, J.; Siddique, S.; Islam, M. H.; Ali, H.; Gupta, K. D.; and George, R. 2025. Advanced Tool Learning and Selection System (ATLASS): A Closed-Loop Framework Using LLM. *arXiv preprint arXiv:2503.10071*.
- Jimenez, C. E.; Yang, J.; et al. 2023. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *arXiv preprint arXiv:2310.06770*.
- Kim, S.; Moon, S.; Tabrizi, R.; Lee, N.; Mahoney, M. W.; Keutzer, K.; and Gholami, A. 2024. An LLM Compiler for Parallel Function Calling. In *International Conference on Machine Learning (ICML)*.
- Lieret, K.; Yang, J.; and agent Team, S. 2025. mini-SWE-agent: The 100-Line AI Agent for Software Engineering. GitHub repository. <https://github.com/SWE-agent/mini-swe-agent>.
- Liu, H.; Ming, Y.; Joty, S.; and Zhao, C. 2026. Harnessing LLM Agents with Skill Programs. *arXiv preprint arXiv:2605.17734*.
- Lumer, E.; Sen, S.; Paul, K.; and Subbiah, V. K. 2026. Recursive Agent Harnesses. *arXiv preprint arXiv:2606.13643*.
- Microsoft. 2026. Agents Powered by the GitHub Copilot Harness: Overview. Microsoft Copilot Studio documentation, <https://learn.microsoft.com/en-us/microsoft-copilot-studio/agents-experience/overview>. Accessed: 2026-08-26. The GitHub Copilot harness runs on top of the GitHub Copilot CLI and SDK, which own the agent loop, tool invocation, planning, and session state.
- OpenAI. 2024. Introducing SWE-bench Verified. OpenAI Blog. <https://openai.com/index/introducing-swe-bench-verified/>.
- OpenAI. 2026a. Codex as a Platform: Build on the Open Agent Harness. <https://developers.openai.com/blog/codex-as-a-platform>. Accessed: 2026-08-26. Repositions the open-source Codex CLI / SDK / app-server as an agent harness for embedding into non-coding products (operations, support, security, and business-workflow applications).
- OpenAI. 2026b. Programmatic Tool Calling. OpenAI API developer documentation.
- Patel, I.; Sen, S.; Lumer, E.; and Subbiah, V. K. 2026. The Bitter Lesson of Tool Calling. *arXiv preprint arXiv:2608.06370*.
- Patil, S.; Zhang, T.; Wang, X.; and Gonzalez, J. 2023. Gorilla: Large Language Model Connected with Massive APIs. *arXiv preprint arXiv:2305.15334*.
- Patil, S. G.; Mao, H.; Yan, F.; Ji, C. C.-J.; Suresh, V.; Stoica, I.; and Gonzalez, J. E. 2025. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *International Conference on Machine Learning (ICML)*.
- Qi, J.; Fu, Z.; Gao, J.; Zhang, W.; Yan, H.; Wu, X.; and Zhao, X. 2026. LLM-as-Code: Agentic Programming for Agent Harness. *arXiv preprint arXiv:2606.15874*.
- Qian, C.; Han, C.; Fung, Y. R.; Qin, Y.; Liu, Z.; and Ji, H. 2023. CREATOR: Tool Creation for Disentangling Abstract and Concrete Reasoning of Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 6922–6939.
- Schick, T.; Dwivedi-Yu, J.; et al. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761*.
- Vidgen, B.; Mann, A.; Fennelly, A.; Stanly, J. W.; Rothman, L.; Burstein, M.; Benckek, J.; Ostrofsky, D.; Ravichandran, A.; Sur, D.; Venugopal, N.; et al. 2026. APEX-Agents. *arXiv preprint arXiv:2601.14242*.
- Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. *arXiv preprint arXiv:2305.16291*.
- Wang, X.; Chen, Y.; Yuan, L.; Zhang, Y.; Li, Y.; Peng, H.; and Ji, H. 2024. Executable Code Actions Elicit Better LLM Agents. *arXiv preprint arXiv:2402.01030*.
- Wang, X.; Li, B.; Song, Y.; Xu, F. F.; Tang, X.; Zhuge, M.; Pan, J.; Song, Y.; Li, B.; Singh, J.; Tran, H. H.; Li, F.; Ma, R.; Zheng, M.; Qian, B.; Shao, Y.; Muennighoff, N.; Zhang, Y.; Hui, B.; Lin, J.; Brennan, R.; Peng, H.; Ji, H.; and Neubig, G. 2025. OpenHands: An Open Platform for AI Software Developers as Generalist Agents. In *International Conference on Learning Representations (ICLR)*.
- Wang, Z.; Fried, D.; and Neubig, G. 2024. TroVE: Inducing Verifiable and Efficient Toolboxes for Solving Programmatic Tasks. *arXiv preprint arXiv:2401.12869*.
- Wu, Z.; Han, C.; Ding, Z.; Weng, Z.; Liu, Z.; Yao, S.; Yu, T.; and Kong, L. 2024. OS-Copilot: Towards Generalist Computer Agents with Self-Improvement. *arXiv preprint arXiv:2402.07456*.
- Xia, C. S.; Deng, Y.; Dunn, S.; and Zhang, L. 2024. Agentless: Demystifying LLM-based Software Engineering Agents. *arXiv preprint arXiv:2407.01489*.

Xia, C. S.; Wang, Z.; Yang, Y.; Wei, Y.; and Zhang, L. 2025. Live-SWE-agent: Can Software Engineering Agents Self-Evolve on the Fly? *arXiv preprint arXiv:2511.13646*.

Xie, T.; Zhang, D.; Chen, J.; Li, X.; Zhao, S.; Cao, R.; Hua, T. J.; Cheng, Z.; Shin, D.; Lei, F.; et al. 2024. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Xu, F. F.; Song, Y.; Li, B.; et al. 2024. TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks. *arXiv preprint arXiv:2412.14161*.

Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; and Press, O. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv preprint arXiv:2405.15793*.

Yao, S.; Shinn, N.; Razavi, P.; and Narasimhan, K. 2025. τ -bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *International Conference on Learning Representations (ICLR)*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*.

Yuan, L.; Chen, Y.; Wang, X.; Fung, Y. R.; Peng, H.; and Ji, H. 2023. CRAFT: Customizing LLMs by Creating and Retrieving from Specialized Toolsets. *arXiv preprint arXiv:2309.17428*.

A Harness and Prompts

System prompts for all five tool interfaces share a benchmark-domain specific identity paragraph and benchmark task text, and varies in tool lists. Only synthesis and programmatic tool calling add interface-specific guidance (§3). Synthesis encourages packaging reusable steps as parameterized commands but does not require tool creation.

The GitHub Copilot SDK supplies the agent loop, not its default toolset or system prompts, as they got overridden in our benchmark setups.

B Benchmark Details

B.1 TheAgentCompany

TheAgentCompany combines cross-application workflows with local coding and single-application tasks (Figure 2a). Examples include screening resumes from ownCloud against hiring requirements in RocketChat and synchronizing Plane issues with GitLab.

The typed catalog covers task-relevant service APIs and workspace operations, not every shell-accessible endpoint (Table 7). With no canonical benchmark system prompt, we author a shared identity and vary only tools and interface-specific guidance (Appendix A).

Uniform coworker patches (§4.1) recover missed channel messages on the next turn, prevent thread closure while a reply is expected, and reset Redis between tasks to prevent memory carryover. GPT-5 coworkers remain fixed across models and interfaces; scoring follows Table 1.

During development, we observed reward hacking in shell-enabled interfaces: agents extracted answers from evaluator scripts in TheAgentCompany’s public repository rather than completing the intended workflows, or read simulated-coworker files instead of contacting the coworkers. In all reported TheAgentCompany runs, we mitigated these shortcuts by blocking network access to that repository and encrypting the coworker information.

B.2 APEX-Agents

APEX-Agents spans 33 scenario worlds in investment banking, management consulting, and corporate law, with equal task counts across domains. Tasks include contract review, spreadsheet valuation, and survey reconciliation using shared files. Most request written answers; others require creating or editing documents, spreadsheets, or presentations.

The APEX-Agents snapshot combines the application servers in Table 8 with a Unix `sh -c` server. Shell execution enforces filesystem restrictions, environment-variable scrubbing, and execution limits rather than a Python-only restriction. Interface exposure follows §3.

The catalog pins Archipelago revision on June 5, 2026, recording tool names, descriptions, JSON schemas, and the environment-image identifier. It predates the July 16 eleven-server revision’s FMP and EDGAR SEC integrations, avoiding FMP’s paid, external-API-key online mode.

The prompt adapts Archipelago’s reference agent with short domain-specific instructions, but inference uses our harness. The Archipelago-derived judge evaluates each criterion from the task, final response, and workspace-file changes; score is the fraction satisfied, and passing requires all criteria.

Reasoning effort stays at the provider default across interfaces for each model. The public APEX leaderboard instead uses maximal reasoning and the latest server snapshot, so absolute scores are not directly comparable.

C Additional Results

C.1 Scores by Domain

We examine how differences between interfaces vary across enterprise domains or scenarios. The domain breakdown separates TheAgentCompany job families and APEX-Agents professional domains (Figure 6). TheAgentCompany’s smallest families are pooled using effort weighting in the *other* domain.

Every domain follows the score ordering $BASH \geq BASH+TOOL \geq TOOL-ONLY$, with the greatest separation in software engineering. Corporate Law shows the least sensitivity to interface choice, with all five interfaces closely grouped.

C.2 Paired Score Differences

We examine how much task scores differ between interfaces and how uncertain these differences are. For each benchmark, we average interface score differences over matched task-model pairs from both models, giving each pair equal weight (Table 9).

Service	Tools
Workspace	tac_workspace_read_file, tac_workspace_list_dir, tac_workspace_write_file, tac_workspace_write_xlsx, tac_workspace_write_docx, tac_workspace_write_pptx
Rocket.Chat	rocketchat_search_users, rocketchat_list_channels, rocketchat_get_channel_history, rocketchat_get_direct_history, rocketchat_send_channel_message, rocketchat_send_direct_message, rocketchat_create_channel, rocketchat_update_channel_members, rocketchat_set_channel_role
ownCloud	owncloud_search_files, owncloud_list_directory, owncloud_download_file, owncloud_read_text, owncloud_read_pdf, owncloud_read_xlsx, owncloud_read_docx, owncloud_read_pptx, owncloud_read_csv, owncloud_read_image, owncloud_upload_file, owncloud_create_folder, owncloud_move_or_copy_file, owncloud_create_share_link, owncloud_delete_file
GitLab	gitlab_search_projects, gitlab_list_repository_tree, gitlab_get_file, gitlab_read_files, gitlab_search_code, gitlab_create_branch, gitlab_commit_files, gitlab_list_issues, gitlab_get_issue, gitlab_create_issue, gitlab_update_issue, gitlab_list_merge_requests, gitlab_create_merge_request, gitlab_list_commits, gitlab_list_pipelines, gitlab_create_release, gitlab_create_or_update_wiki_page, gitlab_create_project, gitlab_update_project_settings
Plane	plane_list_workspaces_projects, plane_create_project, plane_delete_project, plane_list_members, plane_list_states, plane_list_cycles, plane_search_issues, plane_get_issue, plane_create_issue, plane_update_issue, plane_get_analytics_summary

Table 7: TheAgentCompany’s 60 typed tools by service: callable functions for PTC; JSON tools for TOOL-ONLY and BASH+TOOL.

Server	Tools
Calendar	calendar, calendar_schema
Chat	chat, chat_schema
Docs	docs, docs_schema
Filesystem	get_directory_tree, get_file_metadata, list_files, read_image_file, read_text_file, search_files
Mail	mail, mail_schema
PDF	pdf, pdf_schema
Sheets	sheets, sheets_schema
Slides	slides, slides_schema

Table 8: APEX-Agents’ 20 typed tools across 8 non-shell servers. Except filesystem, each pairs an action-dispatched tool with a schema tool.

C.3 Shell Execution Details

We extend the Bash Efficiency analysis (§5.4) with model-level shell patterns and limitations of the immediate-retry metric.

Shell syntax differs by model even within the same interface. On APEX-Agents, GPT-5.5 uses more heredoc markers for multiline input and fewer logical-chain markers (&&, ||) than Opus-4.8 under BASH, BASH+TOOL, and BASH+SYNTHESIS (Table 10).

The *Repair* metric counts a retry only when the next shell call after a failure shares the first command (Table 3). It misses returns after intervening steps, which occur under BASH+TOOL on both benchmarks, as in `python → pwd → python`. These sequences follow logged order and show a return to the executable, not a confirmed repair or response to the failure.

C.4 Synthesized Tools and Reuse

We extend the Tool Synthesis analysis (§5.5) by examining tools without typed counterparts and the limits of reuse measurements.

Some synthesized tools have no direct typed-tool counterpart. APEX-Agents examples include financial-modeling scripts and general-purpose `extract_text.py` (Table 12).

Reuse tracking captures observable tool history, not every tool an agent may create. It counts overwritten paths once and classifies a call as cross-task reuse only when the tool was created and invoked in an earlier task; write/update calls are excluded. Transient or deleted tools may be missed.

The success-rate comparison between cross-task tool reuse and fresh inline code is sensitive to argument rejections. Fresh inline code means the first occurrence of exact non-

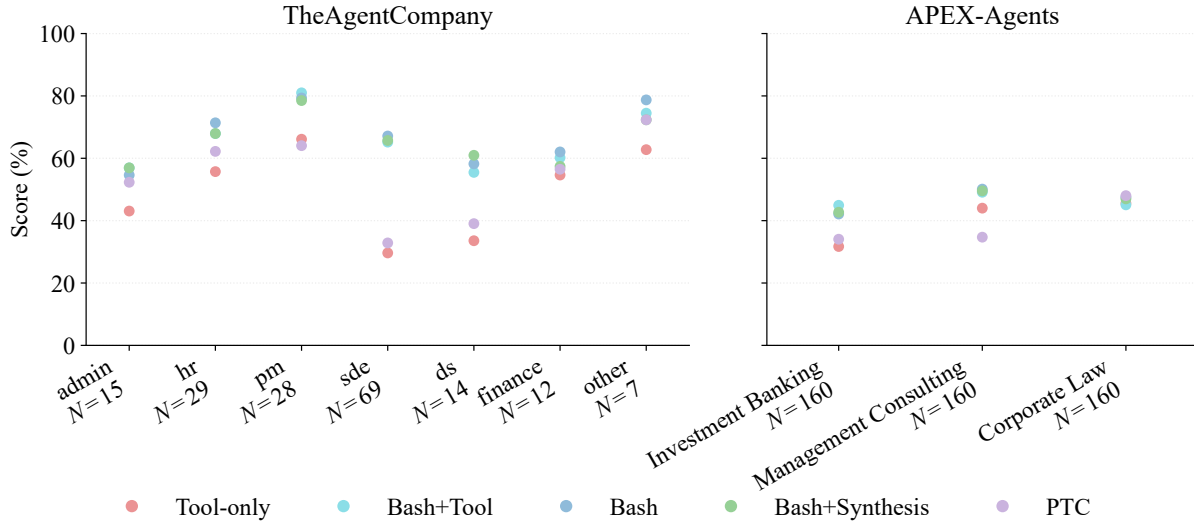


Figure 6: Mean score (%) across models by domain. Dots: interface/domain arms; colors: interfaces; N : tasks/domain. TheAgentCompany’s *other* pools *bm*, *ml*, *qa*, and *research*.

Contrast (A vs. B)	Paired N	Mean score difference (pp, 95% CI)	Wins / Ties / Losses
<i>TheAgentCompany</i>			
BASH+TOOL — BASH	348	-0.6 [-3.0, +1.9]	26 / 282 / 40
BASH+TOOL — TOOL-ONLY	348	+24.1 [+20.1, +28.3]	143 / 192 / 13
BASH — TOOL-ONLY	348	+24.6 [+20.6, +29.0]	161 / 172 / 15
BASH+SYNTHESIS — BASH	348	-1.6 [-3.6, +0.3]	21 / 298 / 29
BASH — PTC	348	+21.2 [+17.0, +25.5]	141 / 188 / 19
PTC — TOOL-ONLY	348	+3.5 [+0.8, +6.2]	49 / 264 / 35
<i>APEX-Agents</i>			
BASH+TOOL — BASH	960	-0.2 [-2.4, +1.9]	149 / 642 / 169
BASH+TOOL — TOOL-ONLY	960	+5.9 [+3.6, +8.3]	208 / 620 / 132
BASH — TOOL-ONLY	960	+6.2 [+3.6, +8.7]	220 / 596 / 144
BASH+SYNTHESIS — BASH	960	-0.2 [-2.4, +2.0]	164 / 638 / 158
BASH — PTC	960	+7.7 [+5.2, +10.2]	228 / 589 / 143
PTC — TOOL-ONLY	960	-1.5 [-3.9, +0.9]	174 / 596 / 190

Table 9: *Mean score difference*: mean paired A minus B in percentage points (positive favors A); brackets: 95% bootstrap CIs. *Paired N*: task-model pairs; *wins / ties / losses*: pair counts summing to N .

library inline code within a task. For GPT-5.5 on APEX-Agents, rejections are more frequent in reuse calls; excluding them leaves only a small success-rate difference. This sensitivity check retains unknown outcomes and measures whole-call status, not individual tool-body reliability.

C.5 Failure Types

We examine how failure types vary across benchmarks and interfaces. An LLM judge assigns each failed trajectory to one failure category (Figure 7).

Failure profiles differ more by benchmark than interface. APEX-Agents failures center on omitted requirements, misapplied domain rules or definitions, and incorrect numbers or formats across all five interfaces. TheAgentCompany spans incomplete deliverables, skipped verification, guessed values, wrong retrievals or deliveries, and environment/access

errors.

TheAgentCompany’s dominant failures vary by interface. TOOL-ONLY fails most often, chiefly through incomplete deliverables or environment/access errors. Shell-based interfaces most often declare completion without verification, while PTC has the highest share of final-message truncation.

Interface	Model	Calls/task	Cmds/call	Composed	Pipe	And/Or	Loop	Heredoc	Multi-line
<i>TheAgentCompany</i>									
BASH	Opus-4.8	11 [7, 16]	4.0 [3, 5]	91.4%	56.4%	51.6%	23.6%	9.9%	51.1%
	GPT-5.5	14 [9, 22]	4.0 [2, 6]	97.3%	51.8%	50.3%	31.5%	33.3%	49.8%
BASH+TOOL	Opus-4.8	6 [3, 11]	3.0 [2, 5]	92.0%	47.9%	71.4%	19.3%	7.4%	31.3%
	GPT-5.5	6 [3, 13]	3.0 [2, 5]	95.7%	37.5%	57.5%	32.2%	32.3%	51.6%
BASH+SYNTHESIS	Opus-4.8	10 [7, 13]	3.0 [2, 4]	84.0%	39.9%	47.1%	23.8%	11.9%	38.7%
	GPT-5.5	18 [12, 27]	3.0 [2, 5]	92.1%	41.8%	59.8%	24.2%	27.5%	36.2%
<i>APEX-Agents</i>									
BASH	Opus-4.8	8 [5, 12]	2.0 [2, 3]	93.9%	32.5%	48.6%	58.3%	16.8%	67.1%
	GPT-5.5	7 [5, 11]	1.0 [1, 2]	99.1%	38.0%	11.6%	77.3%	82.7%	83.8%
BASH+TOOL	Opus-4.8	3 [1, 6]	2.0 [1, 3]	94.7%	22.6%	34.2%	61.5%	34.2%	78.2%
	GPT-5.5	3 [2, 6]	1.0 [1, 1]	99.7%	23.1%	7.5%	86.5%	97.0%	97.2%
BASH+SYNTHESIS	Opus-4.8	10 [6, 16]	2.0 [2, 3]	88.8%	41.9%	28.9%	42.7%	14.8%	47.2%
	GPT-5.5	9 [6, 12]	2.0 [1, 5]	97.9%	56.4%	24.1%	53.6%	53.9%	78.9%

Table 10: Bash metrics for recoverable-command calls by model/interface. *Calls/task*: median per shell-using task; *Cmds/call*: median heuristic static commands (minimum one), excluding embedded code and loop iterations. Brackets: 25th/75th percentiles. Other columns: nonexclusive call percentages per benchmark/model/interface, using source-text markers including embedded code. *Composed*: any listed style, substitution, or redirection with multiple commands. *Pipe*: |, not ||; *and/or*: && or ||; *loop*: for, while, until, if, case, or select; *heredoc*: <; *multi-line*: multiple submitted lines. Markers do not establish execution.

Family	Executables
<i>filesystem</i>	ls, find, pwd, cd, stat, du, df, tree, mkdir, rm, mv, cp, touch, chmod, chown, ln, file, readlink, realpath, basename, dirname, mktemp
<i>text_search_transform</i>	grep, rg, sed, awk, cut, sort, uniq, head, tail, tr, wc, xargs, cat, echo, printf, tee, nl, paste, fmt, fold, tac, rev, expand, unexpand, diff, cmp, patch, iconv
<i>structured_data</i>	jq, yq, sqlite3, duckdb, csvkit, xmllint, xmlstarlet
<i>http_service_api</i>	curl, wget, http, httpie
<i>source_control</i>	git, gh, hg, svn (mostly git in practice)
<i>embedded_program</i>	python, python3, node, ruby, rscript, perl, php, bash, sh, lua, awk (mostly python/python3 in practice)
<i>shell_builtin</i>	test, [, [], read, set, unset, alias, unalias, shift, exit, return, local, declare, typeset, readonly, eval, exec, wait, trap, pushd, popd, dirs, help, let, expr, bc, dc
<i>process_environment</i>	ps, top, htop, kill, pkill, env, export, source, sleep, timeout, which, whereis, date, whoami, hostname, uname, id, groups, printenv, nohup, jobs, disown, true, false
<i>build_test_execute</i>	pytest, npm, npx, yarn, pnpm, make, cmake, cargo, go, mvn, gradle, dotnet, poetry, uv
<i>package_install</i>	pip, pip3, apt, apt-get, dnf, yum, brew, choco
<i>archive</i>	tar, gzip, gunzip, zip, unzip, xz, bzip2, bunzip2, 7z, zstd
<i>document_transform</i>	pandoc, libreoffice, soffice, pdftotext, pdfinfo, tesseract, convert, magick

Table 11: Static command families for Figure 5: heuristic classification by executable; unmatched commands form *other*.

Script	Uses	Substitute
<i>TheAgentCompany</i>		
owncloud_dav.py	122	<i>owncloud_*</i>
rocketchat_cli.py	69	<i>rocketchat_*</i>
rocketchat_dm.py	42	<i>rocketchat_*</i>
rc_users.py	38	<i>rocketchat_*</i>
rocketchat_read.py	24	<i>rocketchat_*</i>
rocketchat.py	19	<i>rocketchat_*</i>
plane_api.py	16	<i>plane_*</i>
pdf_text.py	11	–
rc_channel.py	10	<i>rocketchat_*</i>
pdf_ocr.py	8	–
gitlab_api.py	5	<i>gitlab_*</i>
rc_admin.py	5	<i>rocketchat_*</i>
make_odt.py	3	–
password_validate.py	3	–
pdf_render.py	3	–
<i>APEX-Agents</i>		
extract_text.py	254	–
office_search.py	133	–
pdftext.py	96	<i>pdf_server_*</i>
xlsx_search.py	80	<i>sheets_server_*</i>
xlsx_find.py	61	<i>sheets_server_*</i>
docxtext.py	55	<i>docs_server_*/filesystem_server_*</i>
ocr_pdf.py	53	<i>pdf_server_*</i>
pdfgrep.py	41	<i>pdf_server_*</i>
pptx_text.py	29	<i>slides_server_*</i>
xlsx_dump.py	26	<i>sheets_server_*</i>
rapidocr_pdf.py	24	<i>pdf_server_*</i>
xlsx_recalc.py	23	<i>sheets_server_*</i>
xlsx_update_recalc_read.py	17	<i>docs_server_*/filesystem_server_*</i>
txt2docx.py	13	<i>docs_server_*/filesystem_server_*</i>
xlsx_lo_recalc.py	10	<i>sheets_server_*</i>

Table 12: Top BASH+SYNTHESIS tools ranked by usage. *Uses*: distinct invoking task-model pairs, pooled across models, not raw calls. *Substitute*: closest typed-tool surface (heuristic); dash: no direct counterpart.

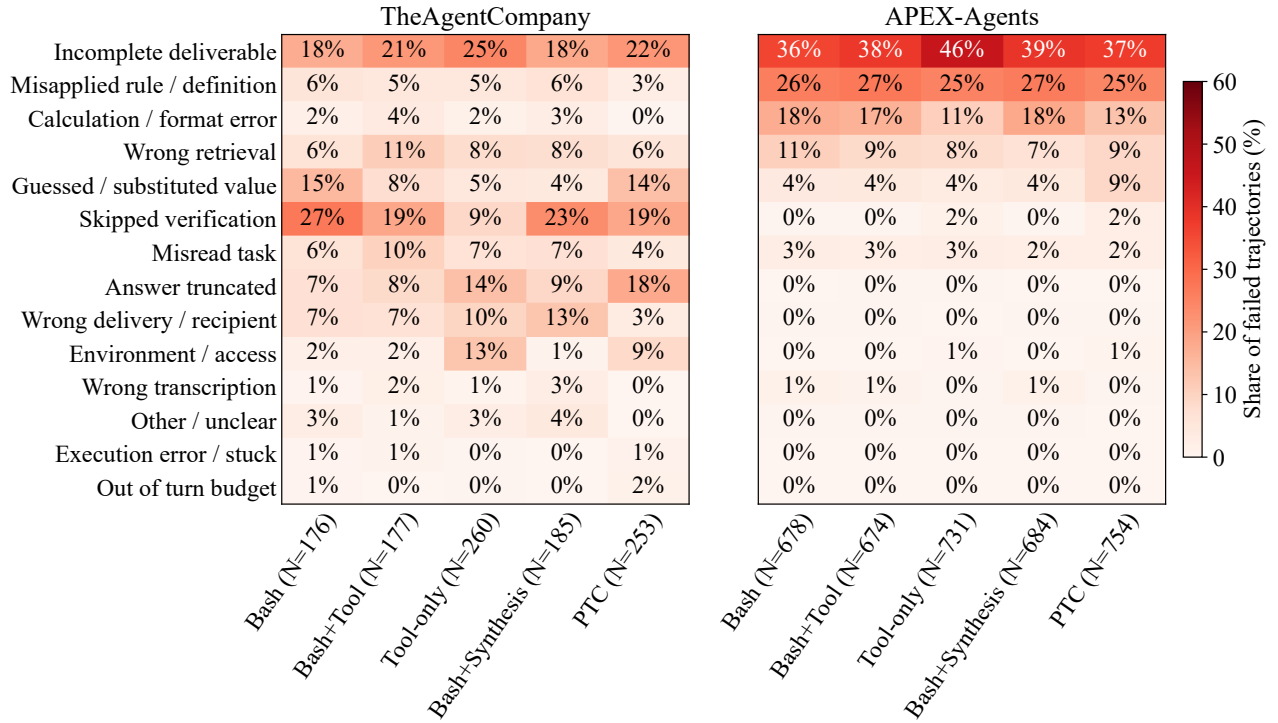


Figure 7: Failure shares among trajectories with score < 1, pooled across models. Rows: 14 root-cause categories ordered by cross-benchmark frequency; columns: benchmark/interface percentages summing to 100%; N : failed trajectories/arm.